

Introduction

Nova.Avalonia.AIKit is a UI toolkit for the interaction patterns around an AI system. It deliberately does not send model requests, store conversations, or require a specific provider SDK. Your application owns orchestration and state; AIKit renders that state and turns user actions into commands and routed events.

What the kit provides

- Conversation controls for prompts, messages, streaming output, citations, and follow-ups.
- Agent controls for progress, reasoning activity, tool calls, task status, and delegation graphs.
- Human-in-the-loop controls for approval, recommendations, feedback, and fine-tuning.
- Code and data controls for diffs, file changes, retrieved context, records, filters, and insights.
- Shared tokens, accessible light and dark resources, localization strings, and motion policy through [NovaTheme](#).

How state flows

Most controls accept ordinary values or [IEnumerableable](#) sources. Bind your view model to those properties, then handle a routed event or command when the user acts. This makes the controls suitable for MVVM as well as code-behind samples.

For example, a host can bind its transcript to [ChatMessageList.ItemsSource](#), append chunks to [StreamingTextBlock.StreamedText](#), and handle [PromptBar.Submitted](#) to start a model turn. AIKit remains independent of how the host obtains the response.

Platform and theme support

The package targets .NET 8 and Avalonia 12.1.1. Its themes layer on top of Avalonia Fluent or Simple, and the gallery demonstrates desktop, browser, Android, and iOS hosts. Motion-capable controls expose local animation switches and also respect the shared motion policy.

How to use these docs

Start with [Getting started](#), then choose a workflow from the [control guide](#). The [Nova.Avalonia.AIKit.Controls](#) API reference contains every property, event, enum, and data item.

Getting Started

Follow these steps to install AIKit, register its resources, and add your first AI control.

Prerequisites

- Avalonia 12.1.1 or later in the Avalonia 12 release line
- .NET 8 or later

Install the package

From your application project, run:

```
dotnet add package Nova.Avalonia.AIKit
```

Register NovaTheme

Add the Nova XML namespace and place **NovaTheme** after your Avalonia base theme. The order matters because AIKit builds on the base theme's resources.

```
<Application xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:nova="https://nova-ui.dev/aikit"
             x:Class="MyApp.App">
  <Application.Styles>
    <FluentTheme />
    <nova:NovaTheme />
  </Application.Styles>
</Application>
```

NovaTheme includes the control themes, design tokens, icons, localized strings, light and dark colors, and the shared motion policy. It can also be placed after **SimpleTheme**.

Add a prompt bar

The same XML namespace exposes all controls:

```
<UserControl xmlns="https://github.com/avaloniaui"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
             xmlns:nova="https://nova-ui.dev/aikit">
  <nova:PromptBar Text="{Binding Draft}"
                 ModelName="Vanilla 1"
                 PlaceholderText="Ask anything..." />
```

```
Submitted="OnPromptSubmitted" />
</UserControl>
```

Handle the routed event when using code behind:

```
private async void OnPromptSubmitted(object? sender, PromptSubmittedEventArgs e)
{
    await _conversation.SendAsync(e.Message);
}
```

For MVVM, bind `Command` to receive the submitted text or `SubmitCommand` to receive the full `PromptSubmittedEventArgs` snapshot. Submission can be rejected by setting `e.Cancel = true`.

Run the gallery

The repository contains interactive samples for every major control:

```
dotnet run --project samples/Nova.Avalonia.AIKit.Gallery.Desktop
```

The `Nova.Avalonia.AIKit.Hermes` sample shows the controls composed into a complete AI workbench:

```
dotnet run --project samples/Nova.Avalonia.AIKit.Hermes
```

Next, browse the [control guides](#) or read about [theming and motion](#).

Theming and Motion

AlKit centralizes control styles, semantic colors, typography, icons, localized strings, and animation policy in **NovaTheme**.

Choose a base theme

Register either Avalonia Fluent or Simple before **NovaTheme**:

```
<Application.Styles>
  <FluentTheme />
  <nova:NovaTheme />
</Application.Styles>
```

```
<Application.Styles>
  <SimpleTheme />
  <nova:NovaTheme />
</Application.Styles>
```

The resources use dynamic color lookups, so switching the application theme variant updates AlKit controls without rebuilding their visual trees.

Override design tokens

Place overrides after **NovaTheme** so they win resource resolution. Prefer semantic Nova keys over editing control templates.

```
<Application.Styles>
  <FluentTheme />
  <nova:NovaTheme />
  <Style>
    <Style.Resources>
      <SolidColorBrush x:Key="NovaAccentBrush" Color="#7C3AED" />
      <SolidColorBrush x:Key="NovaGreenBrush" Color="#059669" />
    </Style.Resources>
  </Style>
</Application.Styles>
```

The theme exposes semantic resources such as **NovaCanvasBrush**, **NovaSurfaceBrush**, **NovaFieldBrush**, **NovaLineBrush**, **NovaInkBrush**, and status brushes. Use them for adjacent application UI to keep custom content visually consistent.

Control motion

Animation-aware controls expose `IsAnimationEnabled`. Disable it on a single control when animation would distract or interfere with deterministic testing:

```
<nova:StreamingTextBlock StreamedText="{Binding Answer}"  
    IsAnimationEnabled="False" />
```

`NovaMotion` provides the shared motion policy used by AKit. Controls also account for visibility and detached visual trees so background animations do not keep doing unnecessary work.

Localization and accessibility

Visible labels and automation text are resource-backed or exposed as properties. Override strings after `NovaTheme` or bind the corresponding text properties for application-specific language. Keep meaningful labels on source, approval, and action controls, and use the API's `AccessibleName` properties where a visual alone would be ambiguous.

Control Guides

AIKit controls are designed to be composed. These guides group them by workflow while the API reference documents each public member.

Guide	Controls
Conversation	<code>ChatMessageList</code> , <code>ChatComposer</code> , <code>StreamingTextBlock</code> , <code>FollowUpList</code>
Prompt bar	<code>PromptBar</code> , <code>PromptCommand</code> , <code>PromptSource</code>
Thinking and loading	<code>ThinkingTrace</code> , <code>LoadingIndicator</code> , thinking item types
Approval and feedback	<code>ApprovalCard</code> , <code>ApprovalQuestion</code> , <code>FeedbackBar</code>
Tool calls	<code>ToolCallList</code> , <code>ToolChip</code> , <code>ToolCall</code> , <code>ToolCallOutput</code>
Agents	<code>AgentAvatar</code> , <code>AgentTaskRow</code> , <code>AgentTimeline</code> , <code>MultiAgentGraph</code>
Code review	<code>CodeBlock</code> , <code>CodeDiffView</code> , <code>CodeFileTree</code>
Context and citations	<code>ContextCard</code> , <code>ContextUsageMeter</code> , <code>CitationChip</code> , <code>SourceChip</code>
Models and input	<code>ModelPicker</code> , <code>InlineSuggestionBox</code> , <code>VoiceWaveform</code> , <code>AttachmentTile</code>
Recommendations	<code>RecommendationCard</code> , <code>ConfidenceMeter</code>
Data and insights	Tables, search, insights, statistics, heatmaps, and navigation
Fine tuning	<code>FineTuneCard</code> , <code>PropertyFieldChip</code> , <code>SelectionToolbar</code>

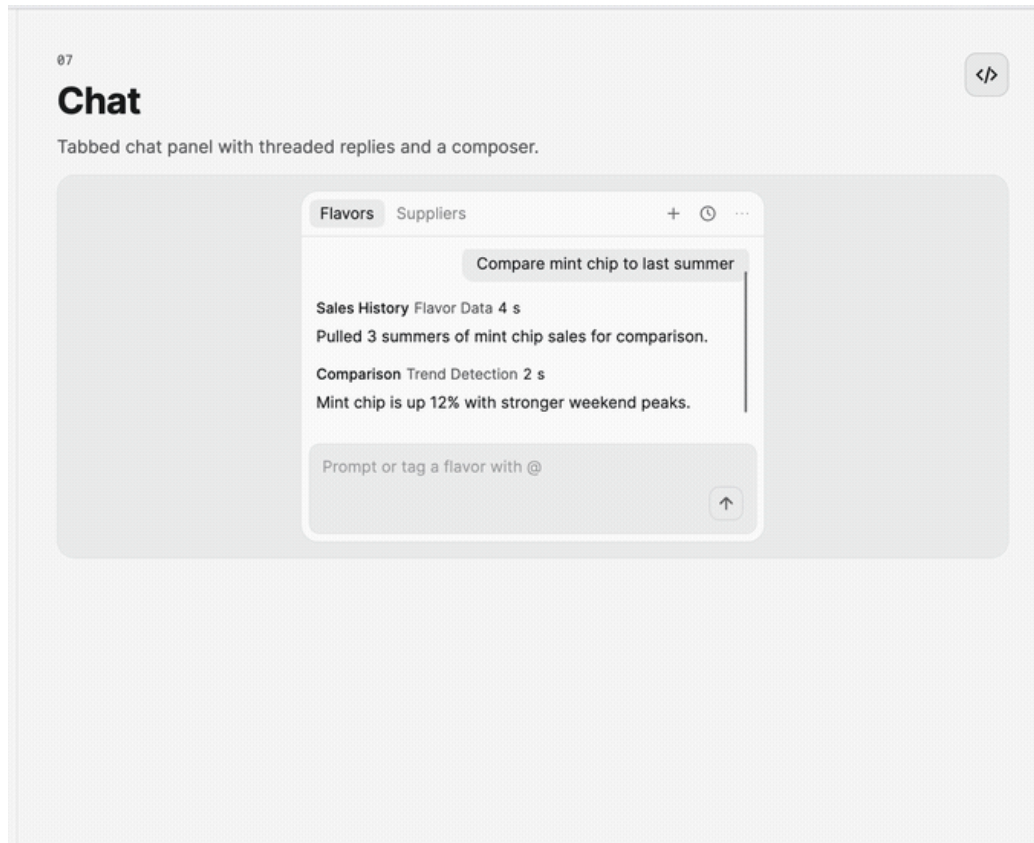
Common integration pattern

1. Bind the control's values or `ItemsSource` to host-owned state.
2. Update that state as provider or agent events arrive.
3. Handle routed events or bind commands for user intent.
4. Send the intent to your application layer; AIKit does not contact a provider itself.

Most collection controls observe `INotifyCollectionChanged`, so an `ObservableCollection<T>` is the natural choice for live transcripts, tools, tasks, and timeline events.

Conversation Controls

Use `ChatMessageList` and `ChatComposer` for a compact chat surface, then add `StreamingTextBlock` and `FollowUpList` when the response arrives incrementally or suggests a next turn.



Display and submit messages

```
<Grid RowDefinitions="*,Auto" RowSpacing="8">
  <nova:ChatMessageList ItemsSource="{Binding Messages}"
    AutoScrollToEnd="True" />
  <nova:ChatComposer Grid.Row="1"
    Text="{Binding Draft}"
    PlaceholderText="Ask anything..."
    Command="{Binding SendMessageCommand}" />
</Grid>
```

`ChatMessageList.ItemsSource` commonly contains `ChatMessage` records. `AutoScrollToEnd` follows new content only while the reader remains near the end; scrolling upward suspends automatic movement. Supply `ItemTemplate` when the transcript requires a custom message presentation.

`ChatComposer.Command` receives the submitted text. `SubmitCommand` and the `Submitted` event receive a richer event snapshot. Set `ClearOnSubmit="False"` when the host should retain the draft.

Submission clears the input before notifying handlers so a handler can supply a replacement draft. If an event handler or command throws synchronously and the input is still empty, the original draft is restored and the exception is propagated. Asynchronous send failures remain the host's responsibility.

Stream an answer

```
<nova:StreamingTextBlock StreamedText="{Binding CurrentAnswer}"  
    RevealDuration="0:0:0.42"  
    TextWrapping="Wrap" />
```

Use the `bound` property when the provider exposes growing snapshots. Replacing or shortening the string resets its inline state. Growing a string for every token copies the accumulated response, so prefer batched `AppendText` calls for long streams:

```
// Run control updates on the Avalonia UI thread.  
answer.ResetStream();  
answer.AppendText("First chunk ");  
answer.AppendText("and the next chunk.");  
answer.FlushStreamedText();
```

`AppendText` reveals new content immediately and debounces the public text snapshot. Call `FlushStreamedText` when the stream finishes, including cancellation when the partial response should be retained. Call `ResetStream` before displaying a new response: it clears text, citations, animations, and the pending snapshot timer even when the previous public snapshot is still empty, and preserves the binding. Cancel the previous producer and reject late chunks from an obsolete turn before resetting. Set `IsAnimationEnabled="False"` for immediate updates without fades.

`AppendText`, `AppendInline`, `FlushStreamedText`, and `ResetStream` must run on the UI thread. Calls from another thread throw before changing stream state; dispatch background producer updates through `Dispatcher.UIThread`.

Streaming Text

Streamed answer with inline sources, actions, and follow-ups.

Pistachio is your fastest-growing flavor, with sales up 23% this month and margins beating vanilla by 8 points.
 scoopdata.io Stone-fruit flavors are trending in the same range, so a seasonal pairing could lift both.

    3 sources

Follow-ups

- ↶ Which flavors sell best in winter
- ↶ Compare gelato and soft serve margins

Offer follow-ups

```
<nova:FollowUpList Header="Try asking"
                  ItemsSource="{Binding FollowUps}"
                  Command="{Binding ChooseFollowUpCommand}" />
```

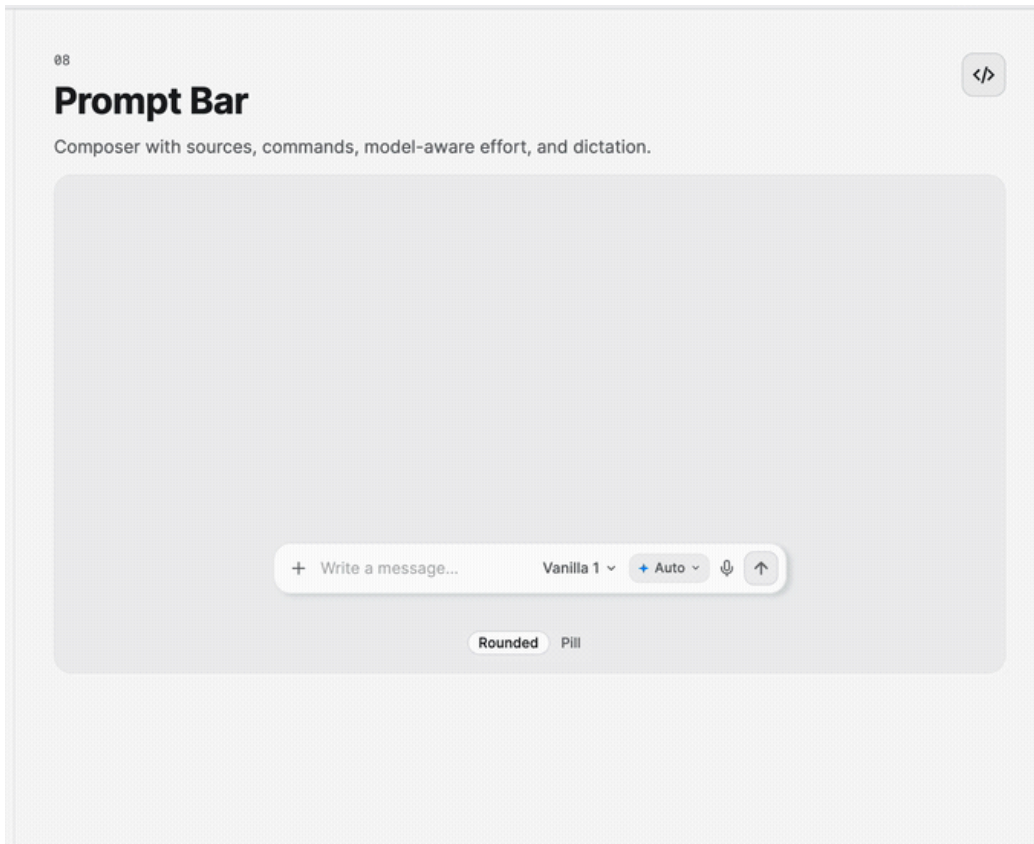
`FollowUpList` raises `FollowUpSelected` and supports `Command` or `FollowUpCommand`. Use `ItemsMaxHeight` to constrain a long generated list.

Relevant API

- [ChatMessageList](#)
- [ChatComposer](#)
- [StreamingTextBlock](#)
- [FollowUpList](#)
- [ChatMessage](#)

PromptBar

PromptBar is a full-featured composer for AI applications. It supports ordinary prompt submission plus searchable commands and sources, structured model selection, reasoning effort, attachment requests, and dictation state.



Basic use

```
<nova:PromptBar Text="{Binding Draft}"
  CommandsSource="{Binding Commands}"
  SourcesSource="{Binding Sources}"
  ModelsSource="{Binding Models}"
  SelectedModel="{Binding SelectedModel}"
  SelectedEffort="{Binding SelectedEffort}"
  TurnId="{Binding TurnId}"
  Submitted="OnSubmitted"
  SourceSelected="OnSourceSelected"
  AttachRequested="OnAttachRequested" />
```

Typing `/` opens command suggestions. Typing `@` opens sources. `MaxSuggestionCount` and `SuggestionsMaxHeight` keep large sources manageable. Collection changes are observed while the control is attached.

Submission contracts

Submitted captures the prompt text, selected model, selected effort, and **TurnId** before the draft is optionally cleared. For commands:

- **Command** receives the submitted text.
- **SubmitCommand** receives **PromptSubmittedEventArgs**.
- **SourceSelectedCommand**, **ModelSelectedCommand**, and **EffortSelectedCommand** receive their corresponding routed-event snapshots.
- **AttachCommand** replaces the built-in source-popup behavior when supplied.

Set **ClearOnSubmit="False"** to keep the text. A handler can reject a submission before side effects by setting **Cancel** on the event arguments.

Dictation

Bind **IsDictating** to the host's recording state. AKit supplies the visual state and start/stop labels; the application remains responsible for microphone permission, capture, and speech-to-text.

Relevant API

- [PromptBar](#)
- [PromptCommand](#)
- [PromptSource](#)
- [PromptSubmittedEventArgs](#)
- [ModelOption](#)

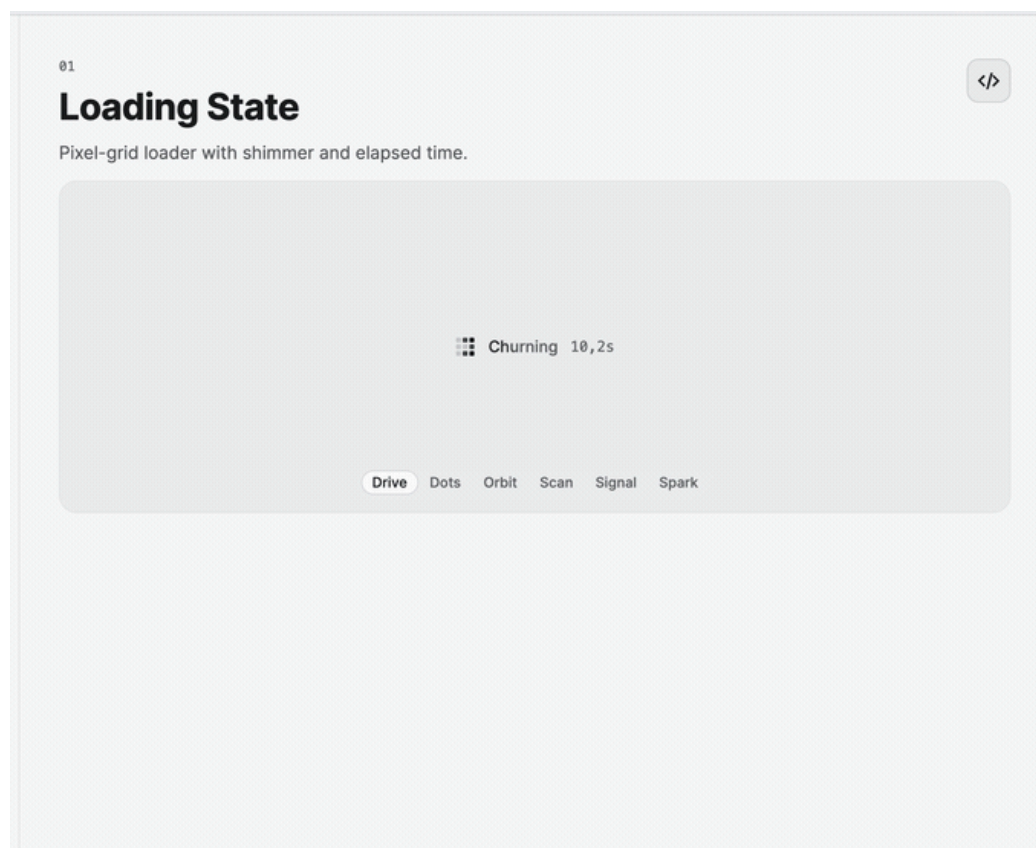
Thinking and Loading

Use `LoadingIndicator` for a compact work state and `ThinkingTrace` when the host has meaningful steps, notes, searches, sources, or tool activity to disclose.

LoadingIndicator

```
<nova>LoadingIndicator Text="Churning"  
    Variant="Drive"  
    IsActive="{Binding IsRunning}"  
    ShowElapsedTime="True" />
```

Choose from `Drive`, `Dots`, `Orbit`, `Scan`, `Signal`, and `Spark`. `ElapsedFormat` customizes the timer label, while `IsAnimationEnabled` can disable movement independently of `IsActive`.

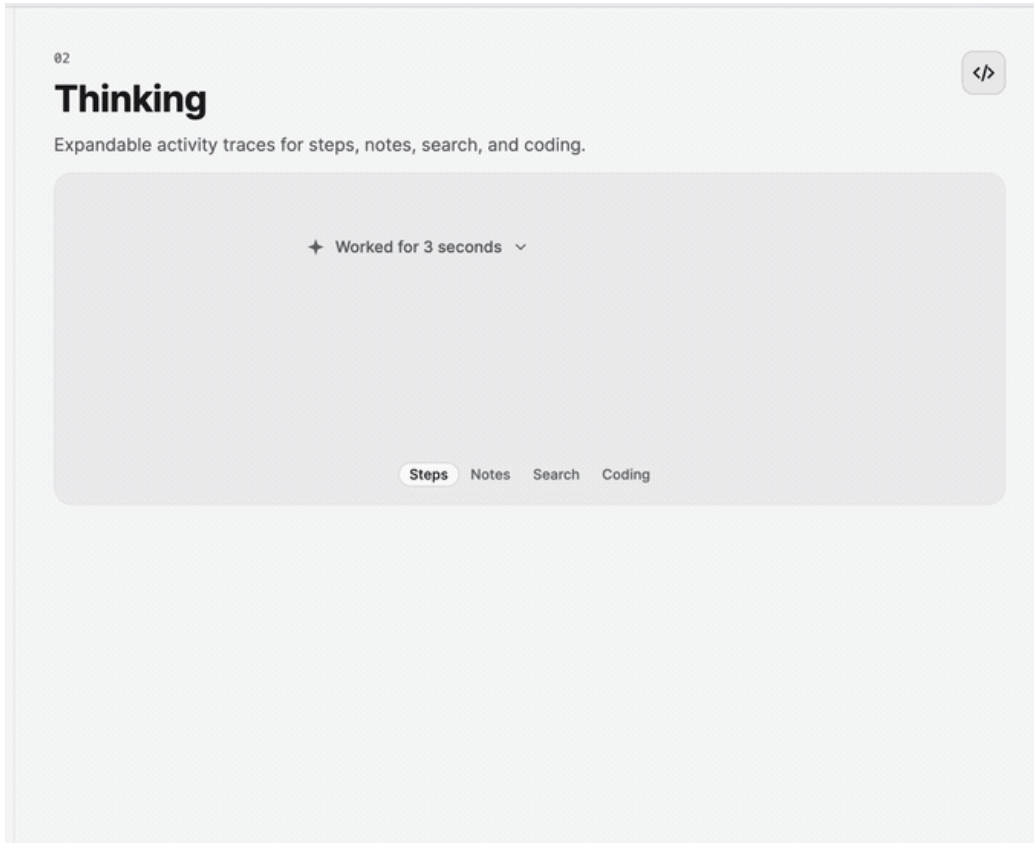


ThinkingTrace

```
<nova>ThinkingTrace ItemsSource="{Binding Activity}"  
    IsThinking="{Binding IsRunning}"  
    IsExpanded="{Binding IsTraceExpanded}"  
    SummaryText="{Binding ActivitySummary}" />
```

Populate the source with `ThinkingStep`, `ThinkingNote`, `ThinkingQuery`, `ThinkingSource`, and `ThinkingToolCall` items. While `IsThinking` is true, the header shimmers and elapsed activity is measured. Once complete, `SummaryText` can replace the active label.

Use `ItemsMaxHeight` for long traces and bind `IsExpanded` two-way when expansion belongs in view-model state.



Relevant API

- [LoadingIndicator](#)
- [LoadingIndicatorVariant](#)
- [ThinkingTrace](#)
- [ThinkingStep](#)

Approval and Feedback

`ApprovalCard` pauses an agent workflow for one or more explicit decisions. `FeedbackBar` captures response actions and evaluation after an answer is shown.

Ask for approval

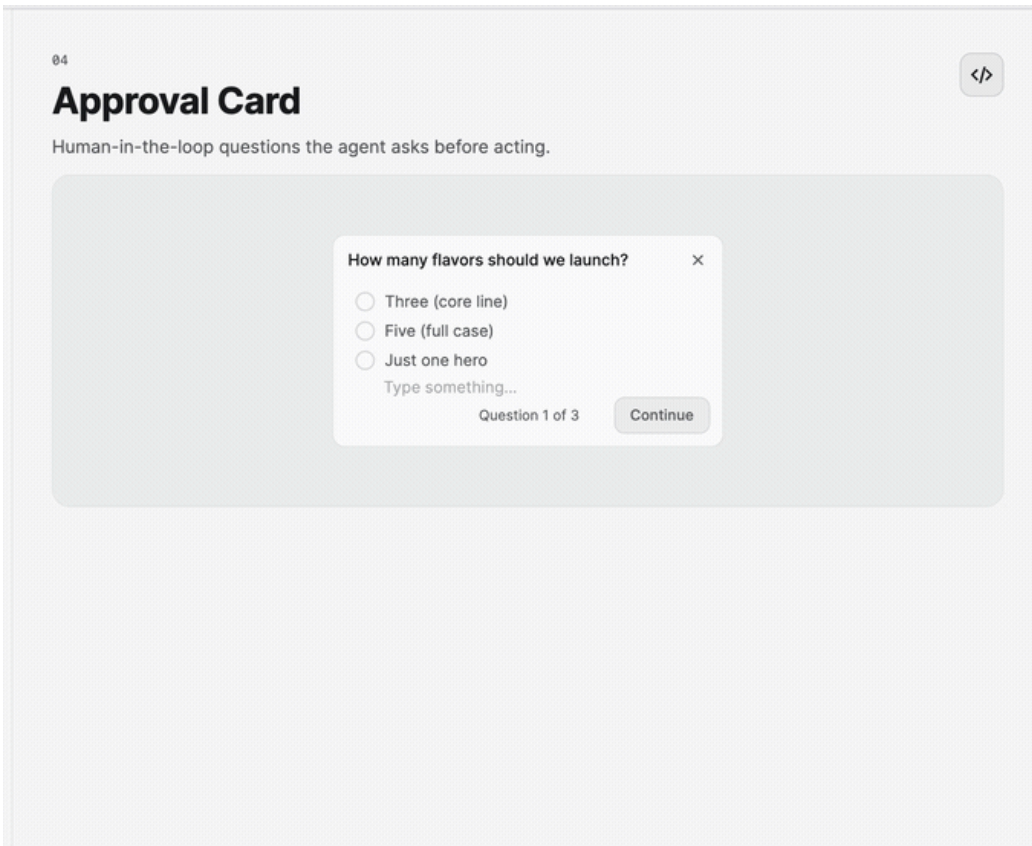
```
<nova:ApprovalCard ItemsSource="{Binding Questions}"
    DraftAnswer="{Binding DraftAnswer}"
    SelectedOptionId="{Binding SelectedOptionId}"
    AnswerCommand="{Binding SubmitApprovalCommand}"
    DismissCommand="{Binding DismissApprovalCommand}" />
```

Each `ApprovalQuestion` can provide predefined `ApprovalOption` values, a risk label, and optional custom-answer support. The card handles question navigation and exposes `CurrentIndex`, `ProgressText`, `CanGoNext`, `CanGoPrevious`, and `CanSubmit` for state-aware templates.

Selecting an option by `SelectedOptionId` also updates `DraftAnswer` to its label. Clearing the ID clears the selected answer and disables submission until a new answer is supplied. A changed draft updates the selected option to match, or becomes a custom answer when allowed. Submissions reject unknown option IDs and reject custom answers when `AllowCustomAnswer="False"`; structured submissions always use the selected option's ID and label together.

For a closed approval such as publishing a change, set `AllowCustomAnswer="False"` and authorize only explicit structured option IDs in the host. Keep custom answers, unknown IDs, and dismissal on the hold path. The Hermes sample keeps the structured answer through its approval gate and applies this policy.

Use `AnswerSubmitted`, `Completed`, `Dismissed`, and `ResetRequested` routed events when code-behind integration is more convenient. Set `CanDismiss="False"` for a required decision.



Capture response feedback

```
<nova:FeedbackBar ResponseId="{Binding ResponseId}"
  TurnId="{Binding TurnId}"
  Revision="{Binding Revision}"
  FeedbackCommand="{Binding SubmitFeedbackCommand}"
  ResponseActionCommand="{Binding ResponseActionCommand}" />
```

The feedback snapshot carries the stable response ID, optional turn ID, and revision so a delayed user action cannot be confused with a newer response. Thumb-down feedback can include a comment. Copy and regenerate have both specific events and a unified `ResponseActionRequested` event.

Changing the response ID, turn ID, or revision closes and discards a pending comment and resets the submitted state. A comment must be started again for the new response. `Reset()` also cancels any pending comment.

Feedback Bar



Copy, regenerate, and thumbs; a thumb down asks what went wrong.

Rocky Road is your weakest flavor this quarter. Consider retiring it in the two lowest regions and reallocating freezer space to Pistachio, which is trending up.



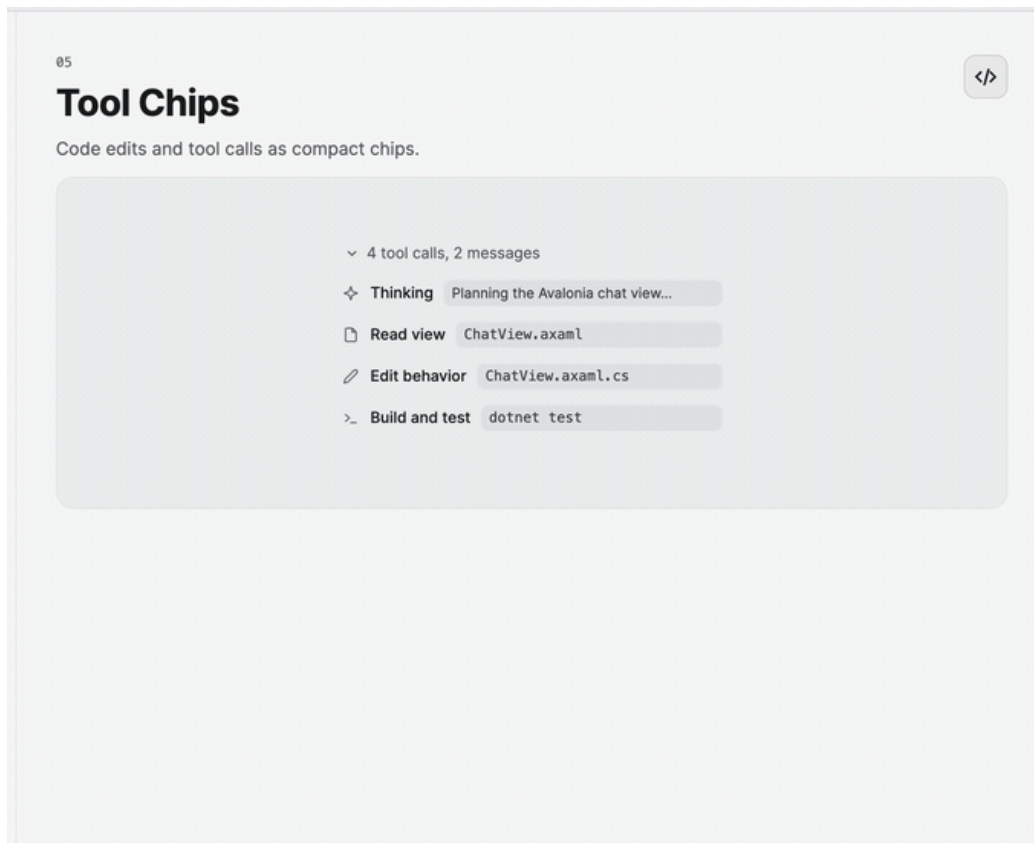
Relevant API

- [ApprovalCard](#)
- [ApprovalQuestion](#)
- [ApprovalOption](#)
- [FeedbackBar](#)
- [FeedbackEventArgs](#)

Tool Calls

`ToolChip` displays a single compact status. `ToolCallList` presents a structured, expandable collection of calls with arguments and output.

```
<StackPanel Spacing="8">
  <nova:ToolChip Text="Read 4 files" />
  <nova:ToolCallList Header="Agent tools"
    ItemsSource="{Binding ToolCalls}"
    IsExpanded="True" />
</StackPanel>
```



Model tool state

Use `ToolCall` items with a `ToolCallKind`, `ToolCallStatus`, arguments, and optional `ToolCallOutput`. Output can represent text or a structured form through `ToolOutputKind`. Update the collection or item state as the orchestrator reports progress.

`ToolCallList` summarizes one or many calls in its header, supports `CallsMaxHeight`, and can animate active or newly added work. Set `IsAnimationEnabled="False"` for static logs or screenshot tests.

`ToolChip.IsMonospaced` is useful for filenames, commands, and code-oriented labels.

Relevant API

- [ToolChip](#)
- [ToolCallList](#)
- [ToolCall](#)
- [ToolCallOutput](#)
- [ToolCallStatus](#)

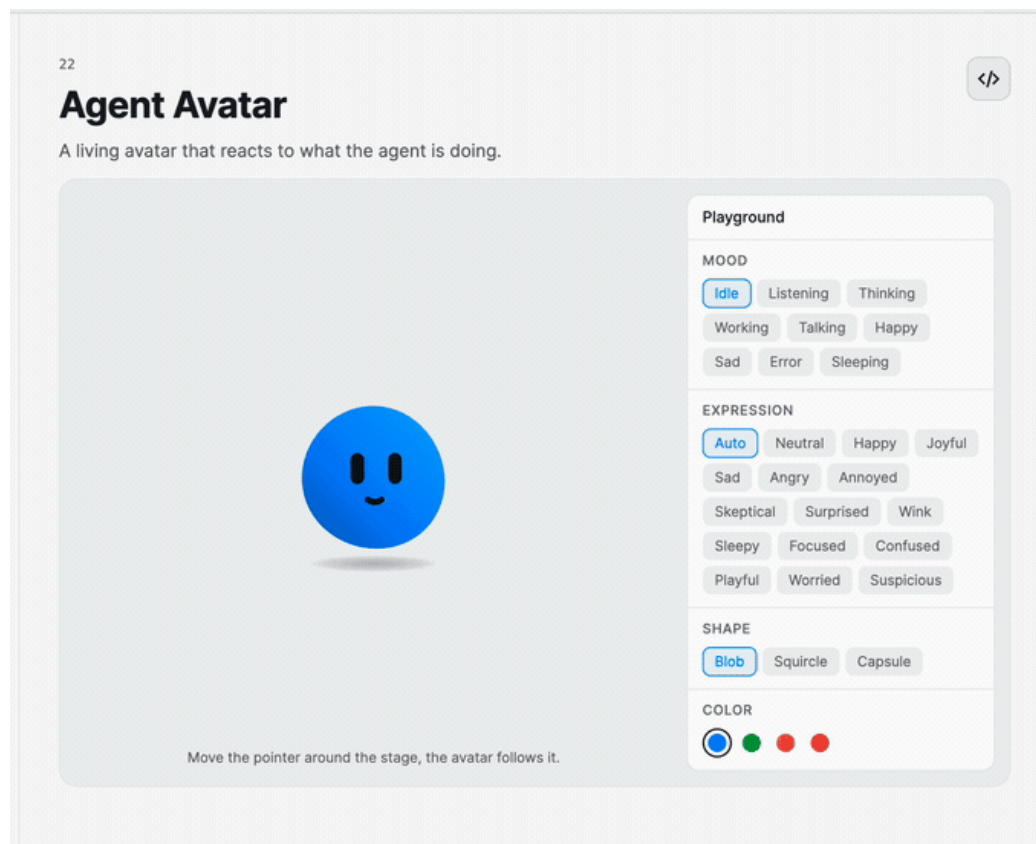
Agent Controls

AIKit includes four levels of agent visualization: identity, a single task, chronological session history, and a graph of collaborating agents.

Identity and task state

```
<StackPanel Orientation="Horizontal" Spacing="12">
  <nova:AgentAvatar Width="72"
    Height="72"
    Mood="Working"
    Expression="Neutral"
    BodyShape="Squircle" />
  <nova:AgentTaskRow Title="Verify vendor records"
    Meta="12 suppliers"
    ProgressText="8"
    Status="Running"
    ItemsSource="{Binding TaskDetails}" />
</StackPanel>
```

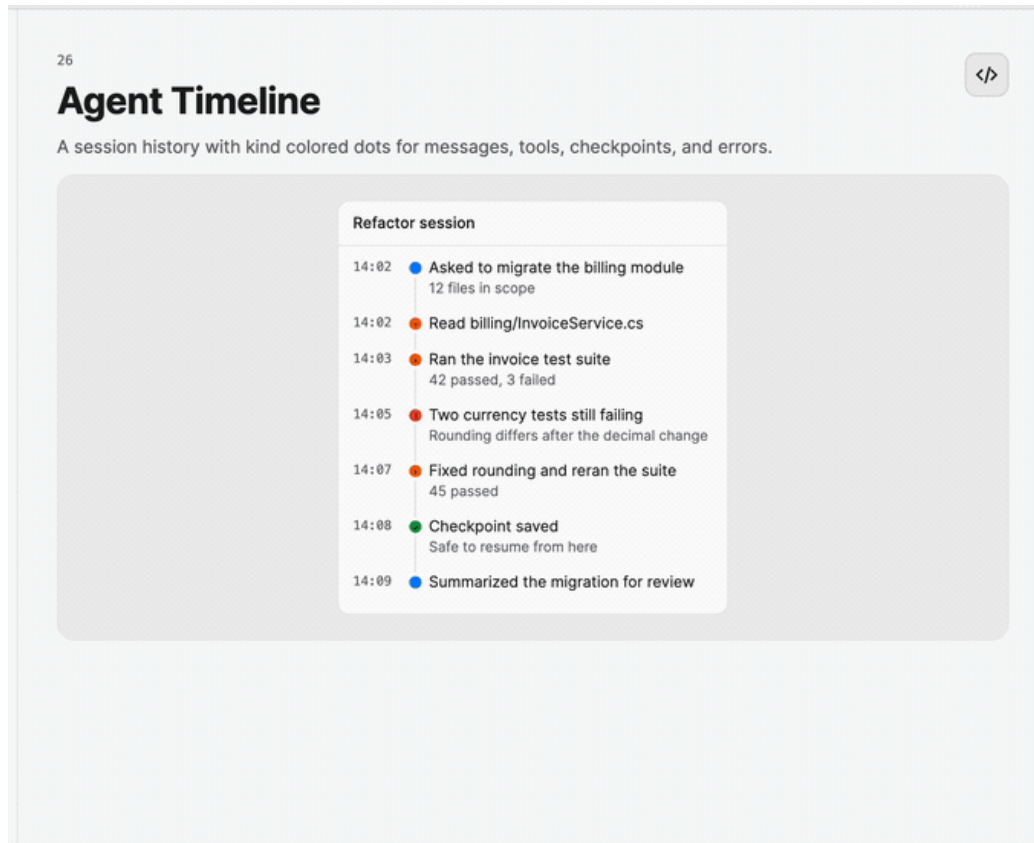
AgentAvatar can track the pointer and animate mood and expression changes. **AgentTaskRow** covers pending, running, completed, failed, and cancelled states, with an expandable detail source.



Session timeline

```
<nova:AgentTimeline Header="Refactor session"
  ItemsSource="{Binding Timeline}" />
```

Supply `TimelineEntry` items categorized by `TimelineEntryKind`. The timeline highlights the last entry and constrains long sessions with `ItemsMaxHeight`.



Multi-agent graph

```
<nova:MultiAgentGraph NodesSource="{Binding Agents}"
  LinksSource="{Binding Delegations}"
  IsAnimationEnabled="True" />
```

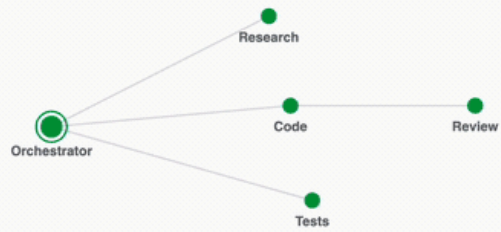
Use `AgentNode` and `AgentLink` objects with stable identities. Running nodes and active links can pulse, while completed, failed, and cancelled states use semantic status colors. Set `AccessibleName` to describe the graph's purpose.

Agent Graph

An orchestrator delegating to subagents, with pulses on active work.



Migration swarm



Relevant API

- [AgentAvatar](#)
- [AgentTaskRow](#)
- [AgentTimeline](#)
- [MultiAgentGraph](#)

Code Review Controls

Use `CodeBlock` for tokenized source, `CodeDiffView` for unified changes, and `CodeFileTree` for a navigable summary of affected files.

Show source code

```
<nova:CodeBlock FileName="AgentService.cs"
                Language="csharp"
                ItemsSource="{Binding CodeLines}"
                CopiedCommand="{Binding CodeCopiedCommand}" />
```

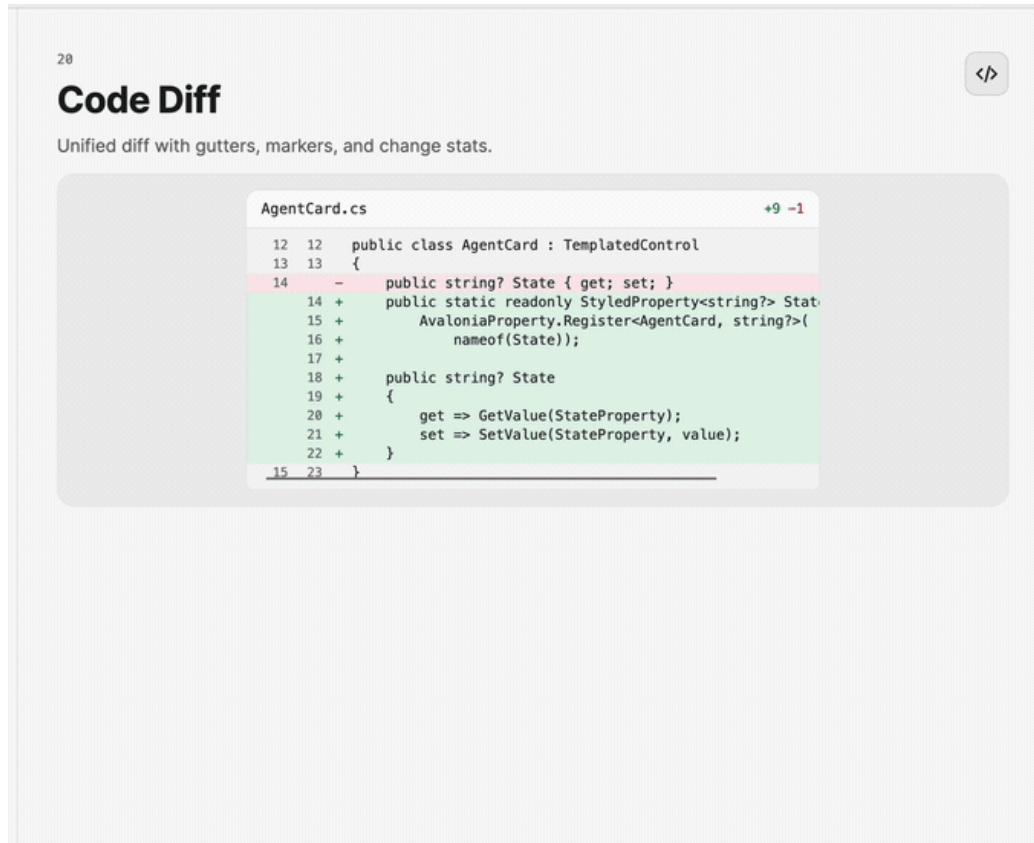
`CodeBlock` accepts `CodeLine` items containing `CodeToken` values. This keeps tokenization provider-agnostic and lets the host stream line-by-line results. Copy success and failure are available as events and commands.



Display a diff

```
<nova:CodeDiffView FileName="AgentService.cs"
                   ItemsSource="{Binding DiffLines}"
                   LinesMaxHeight="420" />
```

Supply `DiffLine` items and classify them with `DiffLineKind`. The control renders gutters, line numbers, add/delete markers, and aggregate change labels.



Navigate changed files

```
<nova:CodeFileTree Header="Files changed"
    ItemsSource="{Binding Files}"
    FileSelectedCommand="{Binding OpenFileCommand}" />
```

`CodeFileItem` captures the path, status, and change counts. Handle `FileSelected` or bind `FileSelectedCommand` to synchronize the tree with a code or diff view.

Code Files



Files the agent touched, with status dots and change counts.

Files changed

src

ChatView.axaml	+38	-12	~
ChatView.axaml.cs	+22	-5	~
AgentStatusCard.cs	+84		+

Themes

Controls.axaml	+31	-8	~
----------------	-----	----	---

tests

PromptBarTests.cs	+46		+
LegacyComposer.cs	-126		-

Relevant API

- [CodeBlock](#)
- [CodeDiffView](#)
- [CodeFileTree](#)
- [CodeLine](#)
- [DiffLine](#)

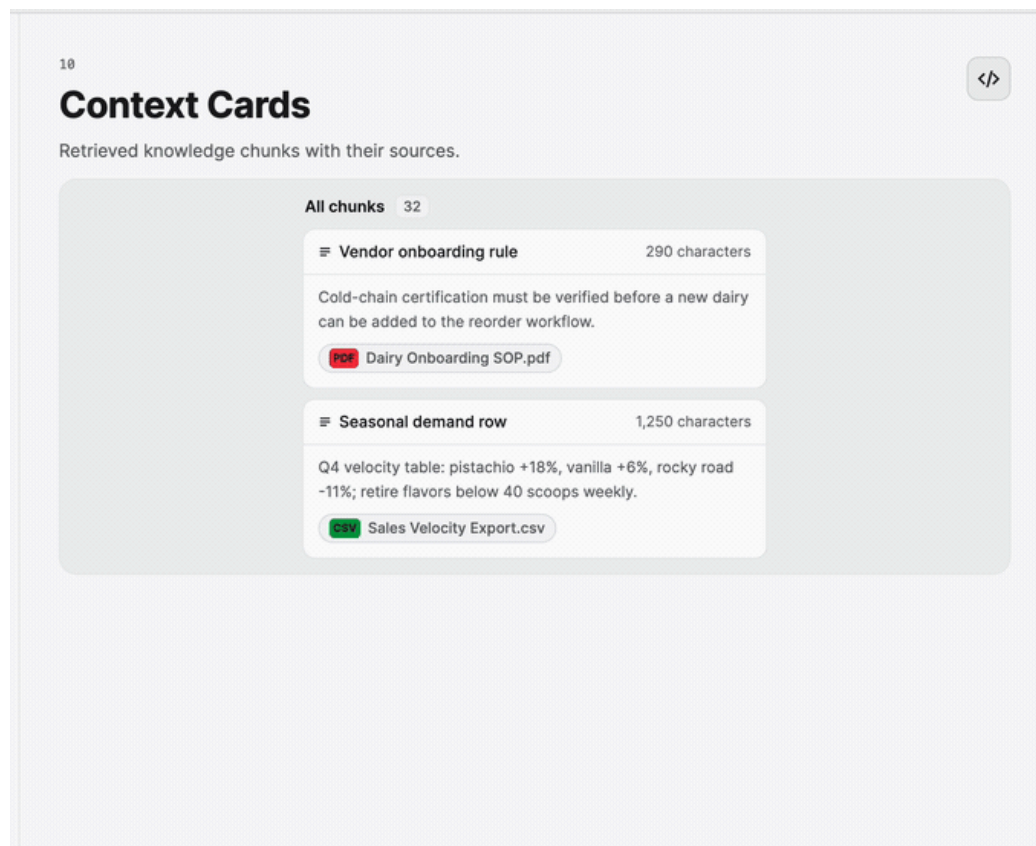
Context and Citations

These controls make grounding visible. Use context cards before or during a run, source and citation chips inside an answer, and the usage meter to show remaining model context.

Retrieved context

```
<nova:ContextCard Title="Refund policy"
  Excerpt="Refunds are available within 30 days..."
  Meta="Updated 2 days ago"
  SourceLabel="Policy"
  SourceName="handbook.md"
  SourceUri="https://example.com/handbook" />
```

`ContextCard` separates the retrieved excerpt from its source metadata and supports custom source foreground and background brushes.



Inline sources

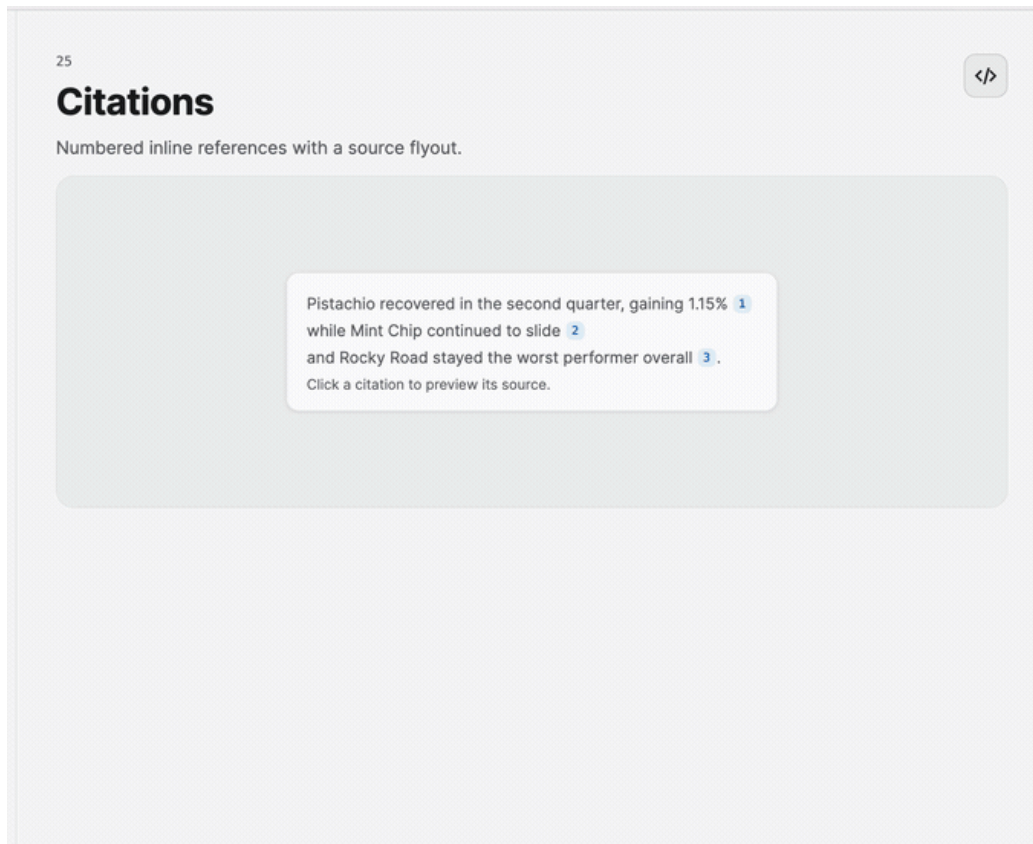
```
<WrapPanel>
  <TextBlock Text="Pistachio recovered in Q2 " />
  <nova:CitationChip Number="1"
    Title="Quarterly sales report"
    Domain="reports.example.com"
```

```

        Snippet="Pistachio closed Q2 at +1.15%."
        NavigateUri="https://example.com/q2" />
<nova:SourceChip Text="Sales report"
        NavigateUri="https://example.com/q2" />
</WrapPanel>

```

`CitationChip` opens a source preview and then navigates to `NavigateUri`. Handle `SourceOpened` and `NavigationFailed`, or bind their commands, for telemetry and error reporting. `SourceChip` is a lighter hyperlink label for a source already explained by surrounding content.



Context-window usage

```

<nova:ContextUsageMeter Capacity="200000"
    UsedTokens="{Binding UsedTokens}"
    Label="Research run"
    WarningThreshold="0.75"
    CriticalThreshold="0.9" />

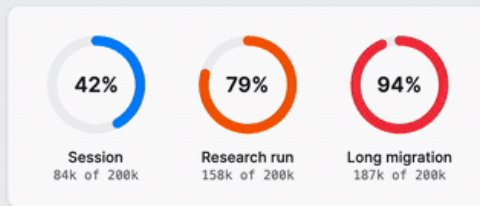
```

The meter computes its percentage, label, and sweep angle. Thresholds change semantic color before the context limit is reached.

Context Meter



Ring gauges for context window usage with warning and critical colors.



Relevant API

- [ContextCard](#)
- [CitationChip](#)
- [SourceChip](#)
- [ContextUsageMeter](#)

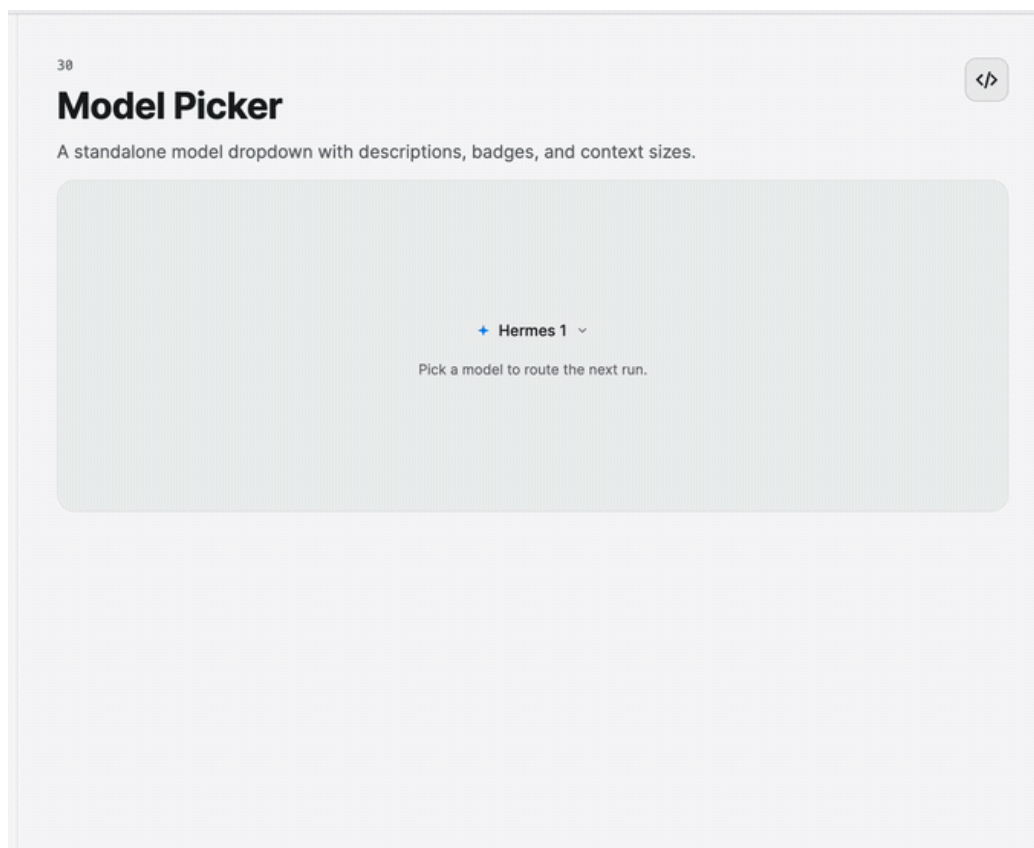
Models and Input

AIKit provides focused input controls that can be used independently of `PromptBar`.

Select a model

```
<nova:ModelPicker ItemsSource="{Binding Models}"
                  SelectedModel="{Binding SelectedModel}"
                  ModelSelectedCommand="{Binding RouteModelCommand}" />
```

Use `ModelOption` entries for stable provider-facing IDs, descriptions, badges, context sizes, and supported `ModelEffortOption` values. `SelectedModel` and `IsDropDownOpen` bind two-way by default.



Offer inline completion

```
<nova:InlineSuggestionBox Text="{Binding Draft}"
                          SuggestionText="{Binding Completion}"
                          PlaceholderText="Ask Nova anything"
                          AcceptSuggestionCommand="{Binding
AcceptCompletionCommand}" />
```

Tab accepts a compatible suggestion and Escape dismisses it. Acceptance is blocked while text is selected or the caret is not at the end, preventing an unexpected edit.

Inline Suggestion

Ghost text completion, accepted with Tab and dismissed with Escape.

Ask Nova anything

Try typing: summarize, translate, write, or explain.

Visualize voice input

```
<nova:VoiceWaveform IsActive="{Binding IsListening}"
    Level="{Binding InputLevel}"
    BarWidth="3"
    BarSpacing="3" />
```

`Level` is normalized from 0 to 1. The host owns audio capture and permissions; this control only renders the input state.

Present attachments

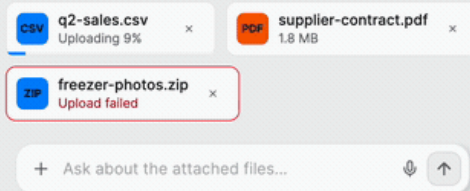
```
<nova:AttachmentTile FileName="q2-sales.csv"
    Detail="Uploading 42%"
    TypeLabel="CSV"
    Progress="0.42"
    Status="Uploading"
    RemoveCommand="{Binding RemoveAttachmentCommand}" />
```

Use `AttachmentStatus` for uploading, completed, and failed visuals. Reaching progress 1 completes an uploading tile. `IsRemovable` controls whether the remove action is available.

Attachments



Files attached to a prompt, with upload progress and removal.



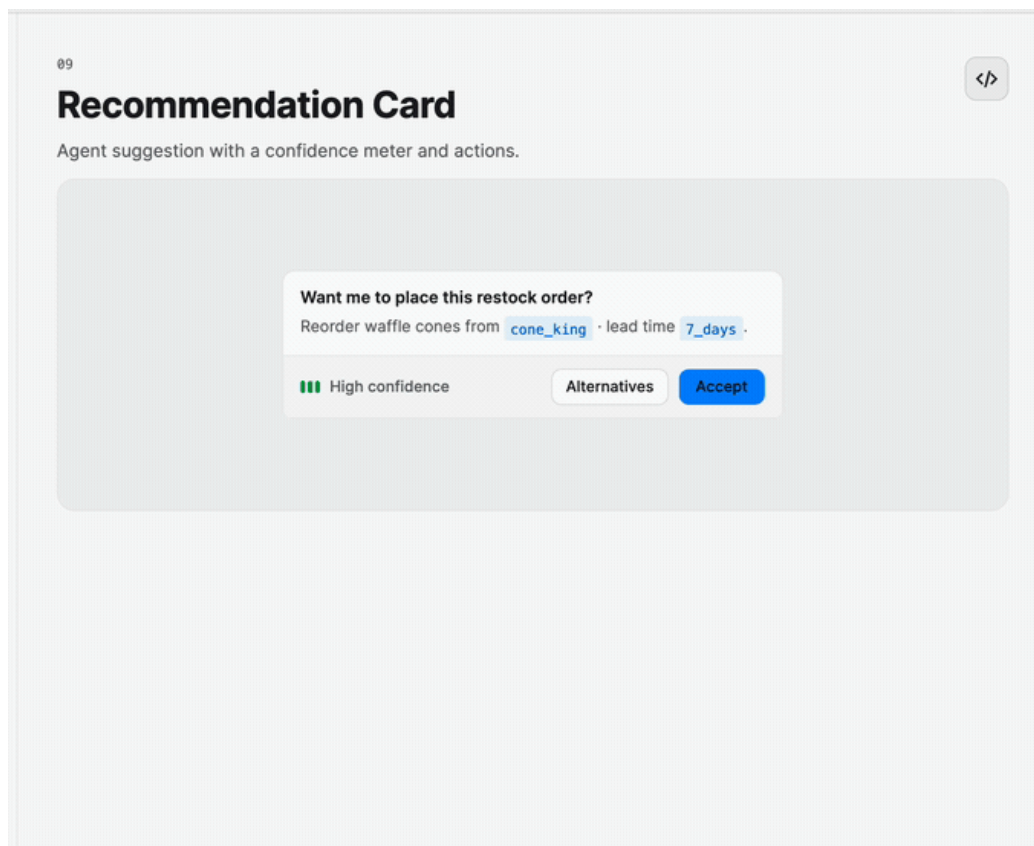
Relevant API

- [ModelPicker](#)
- [InlineSuggestionBox](#)
- [VoiceWaveform](#)
- [AttachmentTile](#)

Recommendations

`RecommendationCard` packages a proposed action with its rationale, confidence, alternatives, and stable orchestration metadata. `ConfidenceMeter` can also be used independently.

```
<nova:RecommendationCard RecommendationId="restock-waffle-cones"
  TurnId="{Binding TurnId}"
  Revision="1"
  Title="Place this restock order?"
  Description="Reorder waffle cones from cone_king."
  Confidence="High"
  AlternativesSource="{Binding Alternatives}"
  AcceptCommand="{Binding AcceptRecommendationCommand}"
  AlternativeSelectedCommand="{Binding
ChooseAlternativeCommand}" />
```



The stable recommendation ID, turn ID, and revision travel with acceptance events, allowing the host to reject stale UI actions safely. `IsAccepted` and `IsAlternativesOpen` make the card state bindable. Use `RecommendationOption` values for alternate actions.

For a standalone semantic indicator:

```
<nova:ConfidenceMeter Level="Medium"
```

```
AccessibleName="Recommendation confidence" />
```

Customize the visible labels with `NoneText`, `LowText`, `MediumText`, and `HighText`.

Relevant API

- [RecommendationCard](#)
- [RecommendationOption](#)
- [ConfidenceMeter](#)
- [ConfidenceLevel](#)

Data and Insights

These controls turn structured model output into reviewable application UI instead of a block of prose.

Review structured changes

```
<StackPanel Spacing="12">
  <nova:DiffTable Header="Proposed changes"
    ItemsSource="{Binding Changes}" />
  <nova:RecordsTable ItemsSource="{Binding Records}" />
  <nova:FilterTable ItemsSource="{Binding FilteredWork}"
    SelectedStatus="{Binding SelectedStatus}" />
</StackPanel>
```

`DiffTableRow` classifies old and new values with `DiffKind`. `CompanyRecord` supports selection, tags, and relationship metadata. `FilterTableItem` and `FilterTableStatus` provide live status filtering; observable sources update the view while attached.

`FilterTable.FilteredItems` exposes the matching rows in source order. The built-in list virtualizes this projection, removing excluded rows immediately so sparse filters remain efficient on large sources. Chip counts still reflect the full source, and status changes update matching rows incrementally.

11

Diff Table

AI proposed edits sweeping through tabular data.

Proposed menu cleanup		
Flavor	Category	Supplier
Rocky Road	• Classic	aurora-scoops
Bubblegum	• Retro	kumo-creamery
Mint Chip	• Classic	maple-orbit

Search and navigation

```

<Grid ColumnDefinitions="240,*">
  <nova:SidebarNav ItemsSource="{Binding Sections}"
    SearchText="{Binding NavigationFilter}"
    ItemSelectedCommand="{Binding NavigateCommand}" />
  <nova:SearchPanel Grid.Column="1"
    ItemsSource="{Binding SearchCorpus}"
    SearchText="{Binding Query}"
    ItemSelectedCommand="{Binding OpenResultCommand}" />
</Grid>

```

`SearchPanel` exposes `FilteredItems` and `IsEmpty`, limits results with `MaxResultCount`, and provides event or command activation. `SidebarNav` groups `SidebarNavItem` objects into sections and includes workspace and new-item actions.

Show generated insights

```

<StackPanel Spacing="12">
  <nova:InsightCards Header="Highlights"
    ItemsSource="{Binding Insights}" />
  <UniformGrid Columns="2">
    <nova:StatTile Label="Sessions" Value="178" />
    <nova:StatTile Label="Tokens" Value="71.5M" />
  </UniformGrid>
  <nova:ActivityHeatmap Values="{Binding DailyUsage}"
    AccessibleName="Daily agent activity" />
</StackPanel>

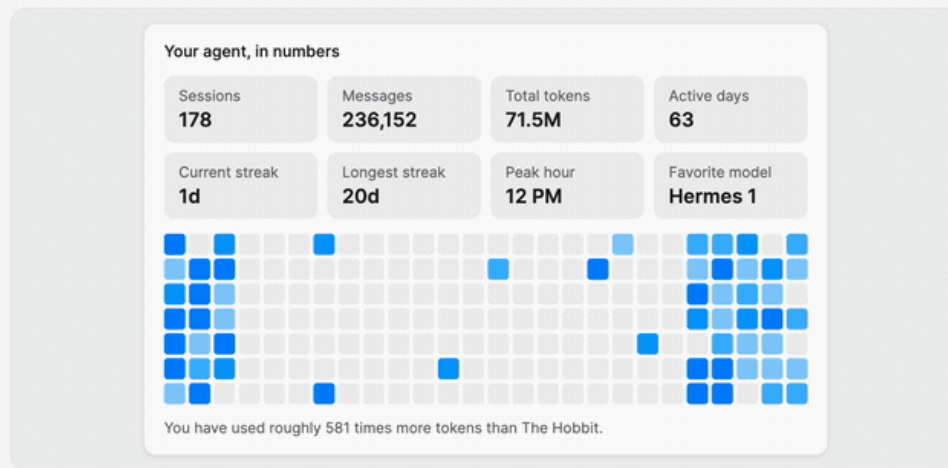
```

`InsightCards` pages through `Insight` objects and their statistics and trend series. `ActivityHeatmap` uses a date-aligned values source, configurable cell sizing, and an accessible summary format.

Usage Stats



Stat tiles and an activity heatmap for usage dashboards.



Relevant API

- [DiffTable](#)
- [RecordsTable](#)
- [FilterTable](#)
- [SearchPanel](#)
- [SidebarNav](#)
- [InsightCards](#)
- [StatTile](#)
- [ActivityHeatmap](#)

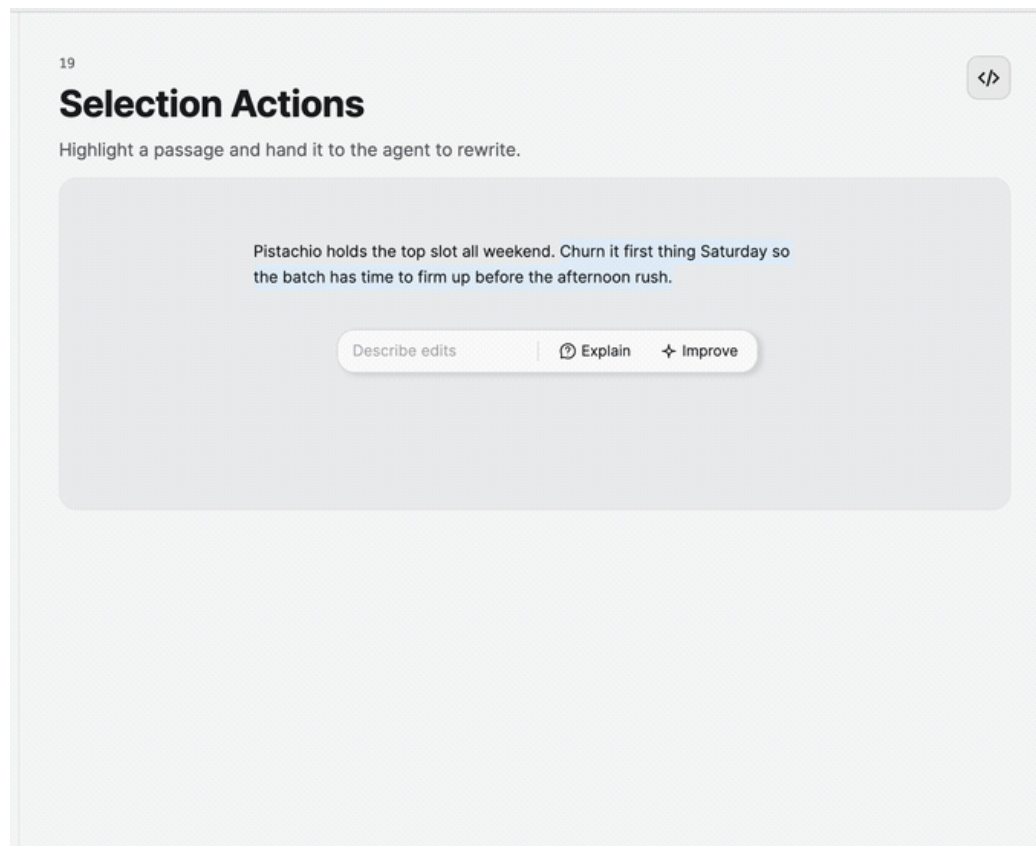
Fine-tuning Controls

Use `SelectionToolbar` for text-oriented AI actions and `FineTuneCard` with `PropertyFieldChip` for compact visual or structured adjustments.

Act on a selection

```
<nova:SelectionToolbar ActionsSource="{Binding SelectionActions}"
  Text="{Binding Instruction}"
  ActionCommand="{Binding RunSelectionActionCommand}"
  SubmitCommand="{Binding SubmitInstructionCommand}" />
```

`SelectionAction` entries describe actions such as rewrite, summarize, or explain. `ActionInvoked` reports the chosen action. The optional text field uses the familiar `CanSend` and `ClearOnSubmit` submission contract.



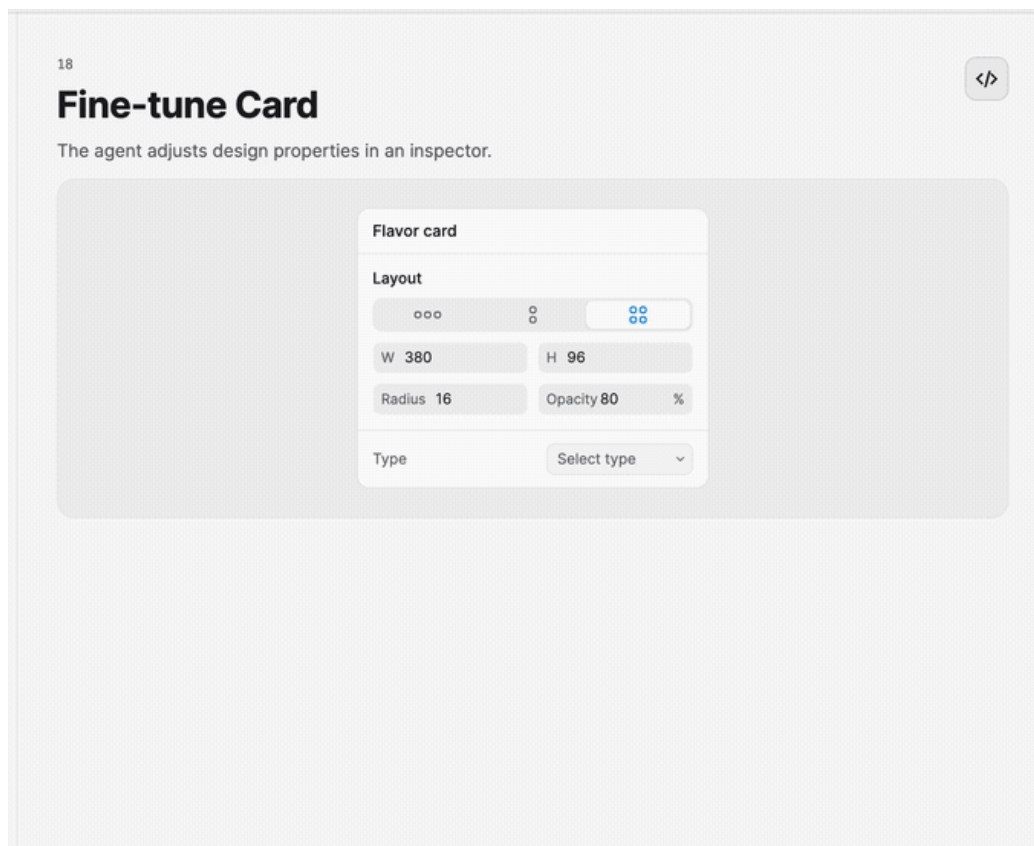
Adjust properties

```
<nova:FineTuneCard Title="Tune selected element"
  Layout="Row"
  ElementWidth="{Binding ElementWidth}"
  ElementHeight="{Binding ElementHeight}"
```

```
ElementRadius="{Binding CornerRadius}"
ElementOpacity="{Binding Opacity}" />
```

`FineTuneCard` provides width, height, radius, and opacity values plus an `IsAdjusting` state. `PropertyFieldChip` is the lower-level numeric primitive when the host needs a custom set of properties:

```
<nova:PropertyFieldChip Label="Temperature"
    Minimum="0"
    Maximum="2"
    Value="{Binding Temperature}"
    Suffix="x" />
```



Relevant API

- [SelectionToolbar](#)
- [SelectionAction](#)
- [FineTuneCard](#)
- [PropertyFieldChip](#)